

A small hybrid language model under a CPU inference budget

DASE7506 Project 1

29 September 2026

Abstract

I trained a causal language model from scratch on the supplied WikiText-2 training split. My model combines an eight-block attention/convolution network, a copy distribution over the current 256-token window, and an order-six modified Kneser–Ney (MKN) model built from training text. A small gate mixes the neural and count probabilities. The model obtains 1.415658 bits per byte (BPB) on the complete test split, compared with 2.102015 for the supplied baseline. On the measured GitHub Actions Linux runner with four threads, CPU FP32 scoring takes 4.64 times the baseline time, with 2.14 GiB peak RAM and 55.81 MB of inference assets.

1 Task and evaluation

The assignment fixes a 2,048-token BPE tokenizer and an evaluator that scores independent causal windows of 256 tokens. At position t , the model predicts x_{t+1} from the observed prefix $x_{\leq t}$, without carrying state between windows. The score is

$$\text{BPB} = \frac{-\sum_t \log_2 p(x_{t+1} \mid x_{\leq t})}{\text{number of raw UTF-8 bytes in the split}}.$$

I used validation for model selection and kept the data, tokenizer and evaluator unchanged. I used no external training text or pretrained weights. I fixed the prediction method before testing. The OpenVINO execution update preserves the weights and prediction rule; I also fixed its files before reproducing the test score. The course limits are five times baseline CPU scoring time, 4 GiB peak evaluation RAM, and 64 MiB of uncompressed inference assets.

2 Model

The neural backbone has width 288 and eight causal blocks. Blocks 1, 3, 5 and 7 use global self-attention; blocks 2, 4, 6 and 8 use gated causal depthwise convolution with kernel width seven. The model also uses rotary positions, RMSNorm, SwiGLU feed-forward layers, and tied token/output embeddings. This layout keeps some global context mixing while replacing half of the attention blocks with cheaper local mixing.

The output head first forms a 2,048-way vocabulary distribution. A separate pointer attends only to input tokens at or before the current position and adds probability to their token IDs. Its triangular mask rules out later input positions. A sparse order-six MKN model supplies another normalized distribution using within-window token history and count tables built from the training split. The gate uses four input-derived confidence features: the neural/copy top log-probability, its top-two margin, the MKN backoff at the highest matched order, and that order’s largest continuation mass. If w_t is its bounded sigmoid output, the combined distribution is

$$p_t = (1 - w_t)p_{\text{neural+copy},t} + w_t p_{\text{MKN},t}.$$

Neither the gate nor the count model sees the target token during inference. I fitted the gate coefficients and feature normalization on training text, then selected the feature subset and slope on validation.

I used R-Drop during training, followed by checkpoint averaging, continuation, output calibration, count-aware training and distillation. The auxiliary training objectives are absent at inference. I store one OpenVINO FP32 feature graph plus a PyTorch head and sparse count tables, avoiding a duplicate backbone in the checkpoint. The runtime processes the graph in blocks of eight independent windows and does not carry state between windows.

3 Experiments and interpretation

To test R-Drop, I compared two runs with seed 17, 7,200 updates, batch size 32, the same sampled-window stream, and 58,982,400 primary next-token presentations each. Both use deep-supervision and future-token auxiliary losses; only the second adds R-Drop. R-Drop performs two stochastic forward passes, so equal target counts do not mean equal training FLOPs.

Model	Validation BPB	Interpretation
Auxiliary losses, no R-Drop	1.464994	Control
Add R-Drop	1.450613	Same network and targets
Hybrid backbone	1.428594	Width also changes

The first pair isolates the addition of R-Drop. The hybrid backbone also changes width and feed-forward capacity, so I cannot attribute its gain to convolution alone. Adding a fixed-weight MKN expert built from training text reduced the hybrid model’s validation BPB from 1.428594 to 1.423206. This supports combining the two distributions, but does not identify which token types benefit.

Continuation, calibration and distillation produced a stronger neural model. Holding that model and the order-six count tables fixed, I compared a fixed mixture weight (1.401288 validation BPB) with the dynamic gate (1.399686). I selected the gate features and slope on validation, so this small gain is not an independent replication. These later steps add training and search cost; they are not an equal-budget comparison with the earlier models. The measurements in `RESULTS.md` support the R-Drop and mixture comparisons, but do not establish that the architecture is optimal.

Complete test split	BPB	Scored targets
Supplied baseline	2.102015	428,405
Hybrid model	1.415658	428,405

My model reduces test BPB by 32.65% relative to the baseline. Its test BPB is 0.015971 higher than its selected validation BPB. The splits may differ in difficulty, and repeated development on one validation split can also make its result optimistic. This comparison cannot separate those explanations. I did not use test results to tune the prediction method.

4 Cost, reproducibility and limitations

I measured CPU inference on a GitHub Actions Linux x86-64 runner with Python 3.12, PyTorch 2.7.1+cpu and OpenVINO 2026.4.0. Both predictors used four PyTorch intra-op and inter-op threads; the feature graph also used four OpenVINO workers. I alternated baseline and model order across three fresh-process repetitions on the complete validation split. The median times were 10.987902 and 51.018586 seconds, respectively: a ratio of 4.643160. Peak RSS was 2,295,070,720

bytes (2.14 GiB). The graph, checkpoint and required inference source files occupy 55.81 MB. These measurements meet the three limits on that runner.

Relative CPU time depends on the host and runtime scheduling. The CPU model and host contention were not recorded, so the reported ratio does not establish compliance on other machines. Windows measured 3.246290 times baseline for the same implementation. The measurement records and reproduction command are in `RESULTS.md` and `code/README.md`; the course evaluator defaults to four threads.

The main neural branch accounts for at least 255,225,110 primary training-target presentations and 7,754.94 seconds of training. The alternate teacher adds training outside that branch. Recorded training time across the experiments, including rejected runs, sums to 14.88 hours. These figures are lower bounds because some setup, evaluation and failed-job costs are missing.

The training procedure in `TRAINING.md` and `code/scripts/reproduce_training.py` covers both teacher branches, checkpoint averaging, training-only counts and gate fitting. The initial model uses 7,200 updates; its successive neural continuations use 4,800, 4,800, 3,600 and 3,600 updates, followed by five output-bias epochs and 3,600 count-aware updates. An alternate teacher uses 7,200 plus 4,800 updates. Distillation uses 1,800 updates with teacher weights 0.55/0.45 and soft/hard loss weights 0.75/0.25. I have checked the combined entry point’s plan and unit tests, but have not rerun the entire training sequence through it. A retrained model needs separate validation and resource measurements.

I record file hashes and test results in `code/results/final-evidence/`. Install the dependencies listed in `code/README.md`. Then run from `code/`:

```
python evaluate.py \
--checkpoint checkpoints/final-model.pt \
--device cpu --precision fp32 --threads 4 \
--split test --output reproduced-test.json
```

The checkpoint and ONNX graph are included in the download. Their SHA-256 hashes and the code commit are recorded in the ZIP manifest. No retraining is needed to reproduce the score.

5 Reuse

The course supplied the baseline, data, tokenizer and evaluator. I used RoPE [1], RMSNorm [2], SwiGLU [3], R-Drop [4], modified Kneser–Ney smoothing [5] and within-context copying [6]. I reused no external pretrained weights or training text.

References

- [1] J. Su et al. [RoFormer: Enhanced Transformer with Rotary Position Embedding](#). arXiv:2104.09864, 2021.
- [2] B. Zhang and R. Sennrich. [Root Mean Square Layer Normalization](#). NeurIPS, 2019.
- [3] N. Shazeer. [GLU Variants Improve Transformer](#). arXiv:2002.05202, 2020.
- [4] X. Liang et al. [R-Drop: Regularized Dropout for Neural Networks](#). NeurIPS, 2021.
- [5] S. F. Chen and J. Goodman. [An Empirical Study of Smoothing Techniques for Language Modeling](#). Computer Speech & Language, 13(4):359–394, 1999.
- [6] S. Merity et al. [Pointer Sentinel Mixture Models](#). arXiv:1609.07843, 2016.